JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class FilePermission

java.lang.Object
    java.security.Permission
        java.io.FilePermission

All Implemented Interfaces:

Serializable, Guard

```
@Deprecated(since="25",
            forRemoval=true)
public final class FilePermission
extends Permission
implements Serializable
```

Deprecated, for removal: This API element is subject to removal in a future version.
This permission cannot be used for controlling access to resources as the Security Manager is no longer supported.

This class represents access to a file or directory. A FilePermission consists of a pathname and a set of actions valid for that pathname.

Pathname is the pathname of the file or directory granted the specified actions. A pathname that ends in "/*" (where "/" is the file separator character, `File.separatorChar`) indicates all the files and directories contained in that directory. A pathname that ends with "/-" indicates (recursively) all files and subdirectories contained in that directory. Such a pathname is called a wildcard pathname. Otherwise, it's a simple pathname.

A pathname consisting of the special token "<<ALL FILES>>" matches **any** file.

Note: A pathname consisting of a single "*" indicates all the files in the current directory, while a pathname consisting of a single "-" indicates all the files in the current directory and (recursively) all files and subdirectories contained in the current directory.

The actions are passed to the constructor in a string containing a list of one or more comma-separated keywords. The possible keywords are "read", "write", "execute", "delete", and "readlink".

The actions string is converted to lowercase before processing.

Since:

1.2

See Also:

Permission, Permissions, PermissionCollection

Constructor Summary

Constructors

Constructor	Description
`FilePermission(String path, String actions)`	**Deprecated, for removal: This API element is subject to removal in a future version.** Creates a new FilePermission object with the specified actions.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
boolean	`equals(Object obj)`	**Deprecated, for removal: This API element is subject to removal in a future version.** Checks two FilePermission objects for equality.
String	`getActions()`	**Deprecated, for removal: This API element is subject to removal in a future version.** Returns the "canonical string representation" of the actions.
int	`hashCode()`	**Deprecated, for removal: This API element is subject to removal in a future version.**

		Returns the hash code value for this object.
boolean	**implies**(Permission p)	**Deprecated, for removal: This API element is subject to removal in a future version.** Checks if this FilePermission object "implies" the specified permission.
PermissionCollection	**newPermissionCollection**(	**Deprecated, for removal: This API element is subject to removal in a future version.** Returns a new PermissionCollection object for storing FilePermission objects.

Methods declared in class Permission

checkGuard, getName, toString

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Constructor Details

FilePermission

```
public FilePermission(String path,
                      String actions)
```

Deprecated, for removal: This API element is subject to removal in a future version.

Creates a new FilePermission object with the specified actions. *path* is the pathname of a file or directory, and *actions* contains a comma-separated list of the desired actions granted on the file or directory. Possible actions are "read", "write", "execute", "delete", and "readlink".

A pathname that ends in "/*" (where "/" is the file separator character, `File.separatorChar`) indicates all the files and directories contained in that directory. A pathname that ends with "/-" indicates (recursively) all files and subdirectories contained in that directory. The special pathname "<<ALL FILES>>" matches any file.

A pathname consisting of a single "*" indicates all the files in the current directory, while a pathname consisting of a single "-" indicates all the files in the current directory and (recursively) all files and subdirectories contained in the current directory.

A pathname containing an empty string represents an empty path.

Implementation Note:

In this implementation, the `jdk.io.permissionsUseCanonicalPath` system property dictates how the `path` argument is processed and stored.

If the value of the system property is set to `true`, `path` is canonicalized and stored as a String object named `cpath`. This means a relative path is converted to an absolute path, a Windows DOS-style 8.3 path is expanded to a long path, and a symbolic link is resolved to its target, etc.

If the value of the system property is set to `false`, `path` is converted to a `Path` object named `npath` after `normalization`. No canonicalization is performed which means the underlying file system is not accessed. If an `InvalidPathException` is thrown during the conversion, this `FilePermission` will be labeled as invalid.

In either case, the "*" or "-" character at the end of a wildcard `path` is removed before canonicalization or normalization. It is stored in a separate wildcard flag field.

The default value of the `jdk.io.permissionsUseCanonicalPath` system property is `false` in this implementation.

The value can also be set with a security property using the same name, but setting a system property will override the security property value.

Parameters:

`path` - the pathname of the file/directory.

`actions` - the action string.

Throws:

`IllegalArgumentException` - if actions is `null`, empty, malformed or contains an action other than the specified possible actions

Method Details

implies

```
public boolean implies(Permission p)
```

Deprecated, for removal: This API element is subject to removal in a future version.

Checks if this FilePermission object "implies" the specified permission.

More specifically, this method returns true if:

- *p* is an instanceof FilePermission,
- *p*'s actions are a proper subset of this object's actions, and
- *p*'s pathname is implied by this object's pathname. For example, "/tmp/*" implies "/tmp/foo", since "/tmp/*" encompasses all files in the "/tmp" directory, including the one named "foo".

Precisely, a simple pathname implies another simple pathname if and only if they are equal. A simple pathname never implies a wildcard pathname. A wildcard pathname implies another wildcard pathname if and only if all simple pathnames implied by the latter are implied by the former. A wildcard pathname implies a simple pathname if and only if

- if the wildcard flag is "*", the simple pathname's path must be right inside the wildcard pathname's path.
- if the wildcard flag is "-", the simple pathname's path must be recursively inside the wildcard pathname's path.

"<<ALL FILES>>" implies every other pathname. No pathname, except for "<<ALL FILES>>" itself, implies "<<ALL FILES>>".

Specified by:

`implies` in class `Permission`

Implementation Note:

If `jdk.io.permissionsUseCanonicalPath` is `true`, a simple `cpath` is inside a wildcard `cpath` if and only if after removing the base name (the last name in the pathname's name sequence) from the former the remaining part is

equal to the latter, a simple `cpath` is recursively inside a wildcard `cpath` if and only if the former starts with the latter.

If `jdk.io.permissionsUseCanonicalPath` is `false`, a simple `npath` is inside a wildcard `npath` if and only if `simple_npath.relativize(wildcard_npath)` is exactly "..", a simple `npath` is recursively inside a wildcard `npath` if and only if `simple_npath.relativize(wildcard_npath)` is a series of one or more "..". This means "/-" implies "/foo" but not "foo".

An invalid `FilePermission` does not imply any object except for itself. An invalid `FilePermission` is not implied by any object except for itself or a `FilePermission` on "<<ALL FILES>>" whose actions is a superset of this invalid `FilePermission`. Even if two `FilePermission` are created with the same invalid path, one does not imply the other.

Parameters:

`p` - the permission to check against.

Returns:

`true` if the specified permission is not `null` and is implied by this object, `false` otherwise.

equals

```
public boolean equals(Object obj)
```

Deprecated, for removal: This API element is subject to removal in a future version.

Checks two FilePermission objects for equality. Checks that *obj* is a FilePermission, and has the same pathname and actions as this object.

Specified by:

`equals` in class `Permission`

Implementation Note:

More specifically, two pathnames are the same if and only if they have the same wildcard flag and their `cpath` (if `jdk.io.permissionsUseCanonicalPath` is `true`) or `npath` (if `jdk.io.permissionsUseCanonicalPath` is `false`) are equal. Or they are both "<<ALL FILES>>".

When `jdk.io.permissionsUseCanonicalPath` is `false`, an invalid `FilePermission` does not equal to any object except for itself, even if they are created using the same invalid path.

Parameters:

`obj` - the object we are testing for equality with this object.

Returns:

`true` if obj is a FilePermission, and has the same pathname and actions as this FilePermission object, `false` otherwise.

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Deprecated, for removal: This API element is subject to removal in a future version.

Returns the hash code value for this object.

Specified by:

`hashCode` in class `Permission`

Returns:

a hash code value for this object.

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(Object)`

getActions

```
public String getActions()
```

Deprecated, for removal: This API element is subject to removal in a future version.

Returns the "canonical string representation" of the actions. That is, this method always returns present actions in the following order: read, write, execute, delete, readlink. For example, if this FilePermission object allows both write and read actions, a call to `getActions` will return the string "read,write".

Specified by:

`getActions` in class `Permission`

Returns:

the canonical string representation of the actions.

newPermissionCollection

```
public PermissionCollection newPermissionCollection()
```

Deprecated, for removal: This API element is subject to removal in a future version.

Returns a new PermissionCollection object for storing FilePermission objects.

FilePermission objects must be stored in a manner that allows them to be inserted into the collection in any order, but that also enables the PermissionCollection `implies` method to be implemented in an efficient (and consistent) manner.

For example, if you have two FilePermissions:

1. `"/tmp/-", "read"`
2. `"/tmp/scratch/foo", "write"`

and you are calling the `implies` method with the FilePermission:

```
    "/tmp/scratch/foo", "read,write",
```

then the `implies` function must take into account both the "/tmp/-" and "/tmp/scratch/foo" permissions, so the effective permission is "read,write", and `implies` returns true. The "implies" semantics for FilePermissions are handled properly by the PermissionCollection object returned by this `newPermissionCollection` method.

Overrides:

`newPermissionCollection` in class `Permission`

Returns:

a new PermissionCollection object suitable for storing FilePermissions.

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.